In [39]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix,ConfusionMatrixDisplay, clas
import warnings
warnings.filterwarnings('ignore')
%matplotlib inline
```

In [2]:
```
df = pd.read_csv("Social_Network_Ads.csv")
```

In [3]:
```
df
```

Out[3]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [4]:
```
df.shape
```

Out[4]:
```
(400, 5)
```

In [5]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   User ID          400 non-null    int64
 1   Gender           400 non-null    object
 2   Age              400 non-null    int64
 3   EstimatedSalary  400 non-null    int64
 4   Purchased        400 non-null    int64
dtypes: int64(4), object(1)
memory usage: 15.8+ KB
```

In [6]:
```
df.describe()
```

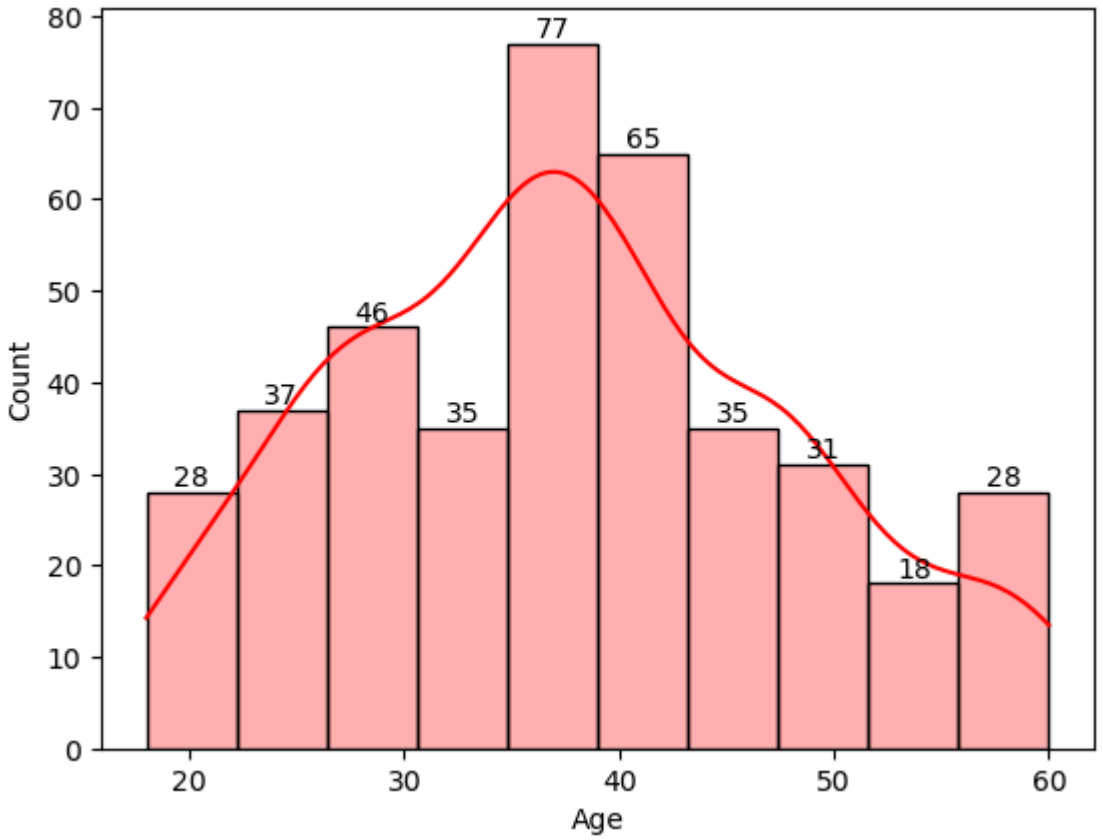
Out[6]:

	User ID	Age	EstimatedSalary	Purchased
count	4.000000e+02	400.000000	400.000000	400.000000
mean	1.569154e+07	37.655000	69742.500000	0.357500
std	7.165832e+04	10.482877	34096.960282	0.479864
min	1.556669e+07	18.000000	15000.000000	0.000000
25%	1.562676e+07	29.750000	43000.000000	0.000000
50%	1.569434e+07	37.000000	70000.000000	0.000000
75%	1.575036e+07	46.000000	88000.000000	1.000000
max	1.581524e+07	60.000000	150000.000000	1.000000

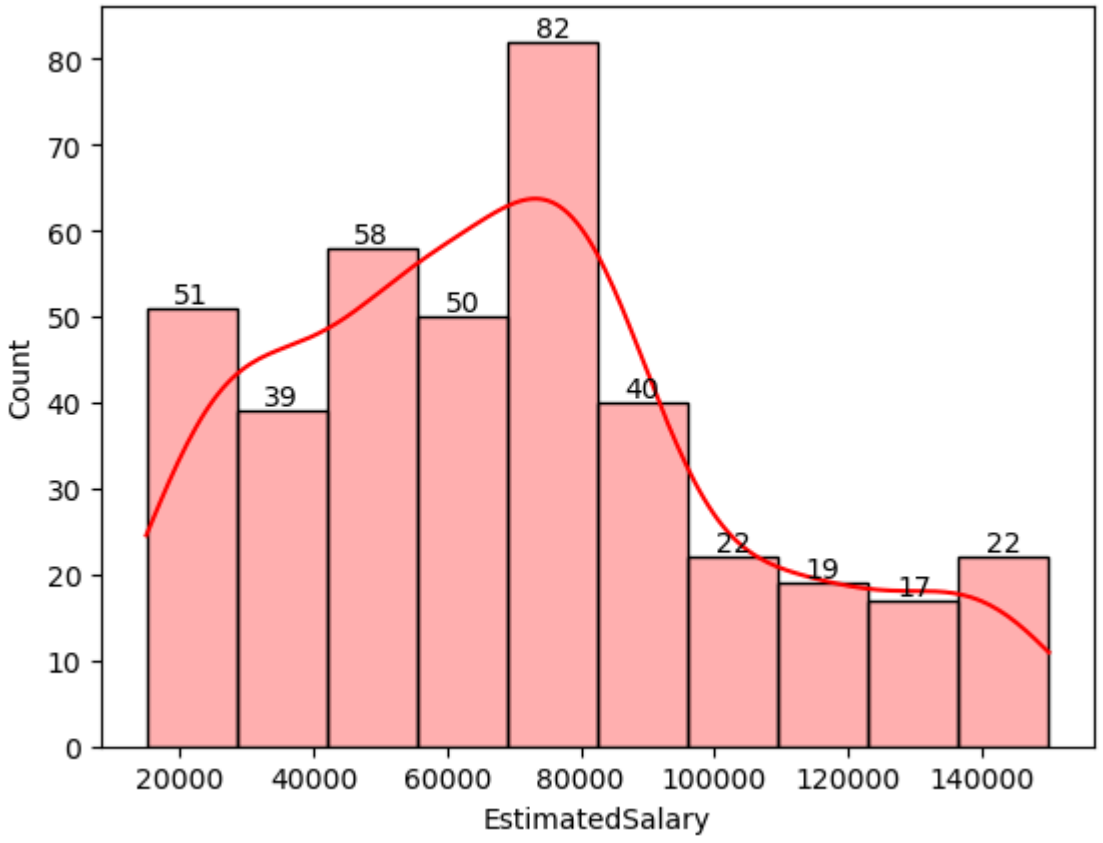
In [7]:
```
df.isna().sum()
```

Out[7]:
```
User ID            0
Gender             0
Age                0
EstimatedSalary    0
Purchased          0
dtype: int64
```

In [9]:
```
histplot = sns.histplot(df['Age'], kde=True, bins=10, color='red', alpha=
for i in histplot.containers:
    histplot.bar_label(i,)
plt.show()
```



In [10]:
```
histplot = sns.histplot(df['EstimatedSalary'], kde=True, bins=10, color='
for i in histplot.containers:
    histplot.bar_label(i,)
plt.show()
```



In [11]:
```
df["Gender"].value_counts()
```

Out[11]:
```
Gender
Female    204
Male      196
Name: count, dtype: int64
```

In [12]:
```
def gender_encoder(value):
  if (value == "Male"):
   return 1
  elif (value == "Female"):
   return 0
  else:
   return -1
```

In [13]:
```
df["Gender"] = df["Gender"].apply(gender_encoder)
df
```

Out[13]:

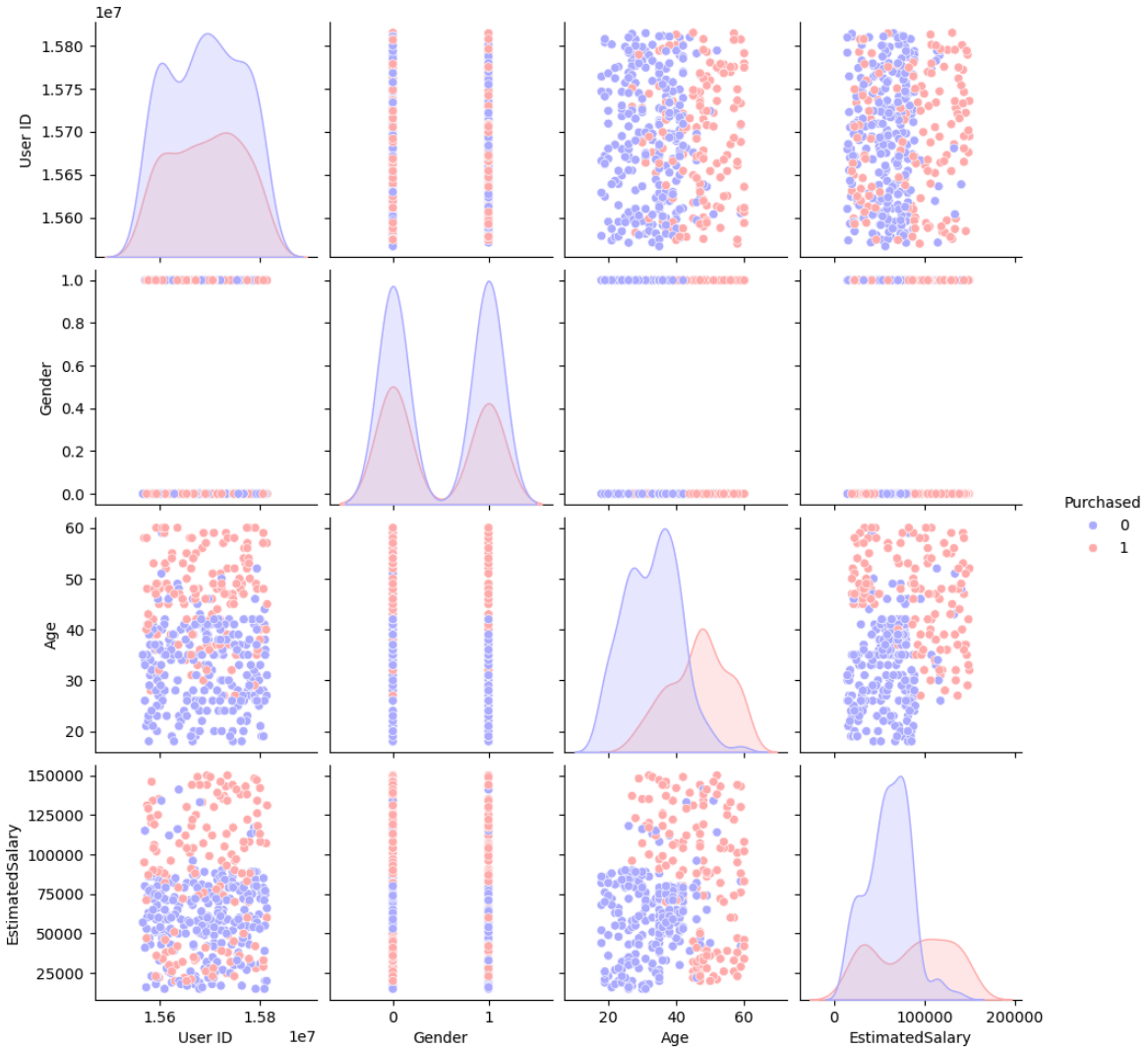
	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	1	19	19000	0
1	15810944	1	35	20000	0
2	15668575	0	26	43000	0
3	15603246	0	27	57000	0
4	15804002	1	19	76000	0
...	...	...	...	...	...
395	15691863	0	46	41000	1
396	15706071	1	51	23000	1
397	15654296	0	50	20000	1
398	15755018	1	36	33000	0
399	15594041	0	49	36000	1

400 rows × 5 columns

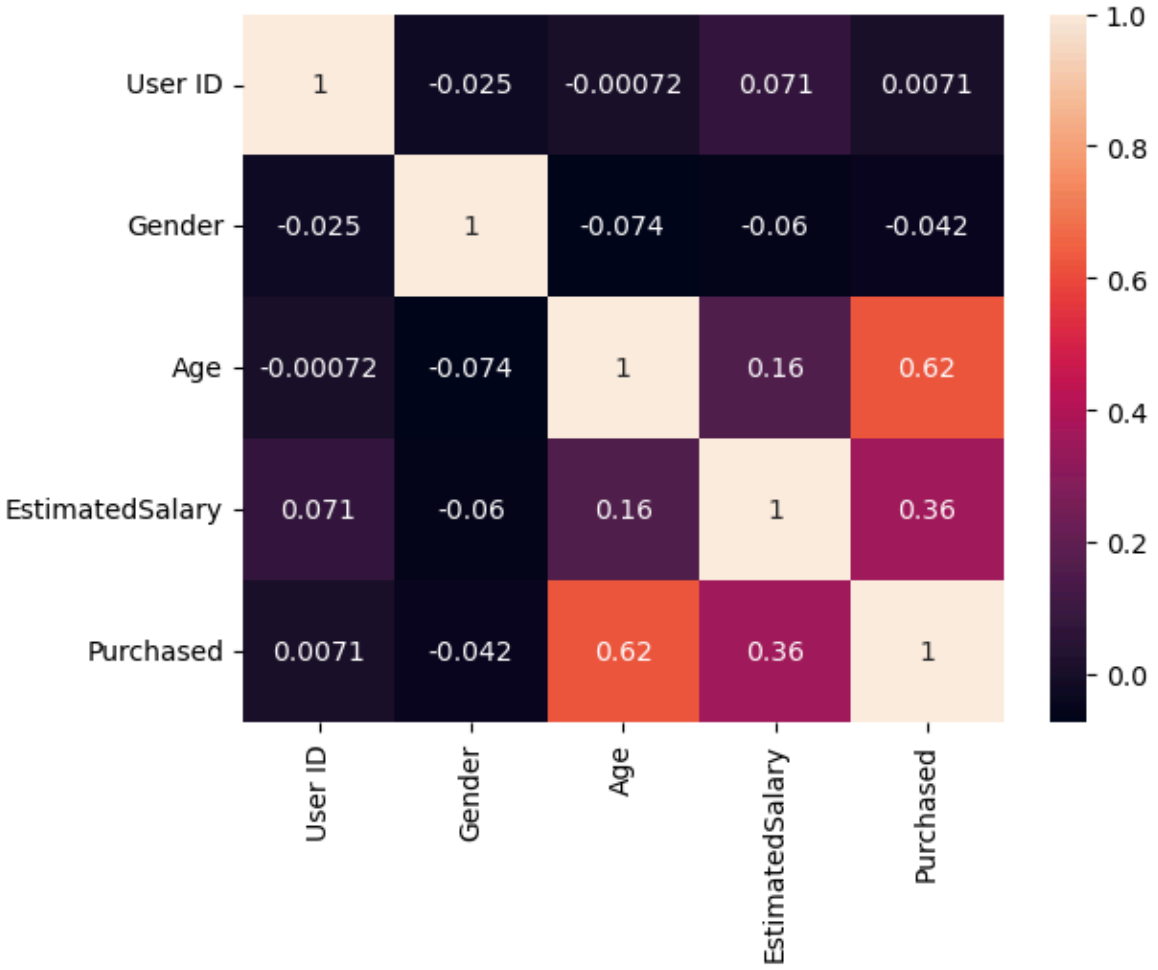
In [14]:
```
df["Purchased"].value_counts()
```

Out[14]:
```
Purchased
0    257
1    143
Name: count, dtype: int64
```

In [15]:
```
sns.pairplot(df,hue='Purchased',palette='bwr')
plt.show()
```



In [16]:
```
sns.heatmap(df.corr(), annot=True)
plt.show()
```



In [17]:
```
x = df[["User ID","Gender","Age", "EstimatedSalary"]]
y = df["Purchased"]
```

In [18]:
```
scaler = StandardScaler()
x = scaler.fit_transform(x)
```

In [19]:
```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
```

In [20]:
```
x_train.shape, x_test.shape, y_train.shape, y_test.shape
```

Out[20]:
```
((320, 4), (80, 4), (320,), (80,))
```

In [21]:
```
model = LogisticRegression()
```

In [22]:
```
model.fit(x_train, y_train)
```

Out[22]:
```
▾ LogisticRegression
▶ Parameters
```

In [23]:
```
y_pred = model.predict(x_test)
y_pred
```

Out[23]:
```
array([0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0,
       0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0])
```

In [24]:
```
model.score(x_train,y_train)
```

Out[24]:
```
0.821875
```

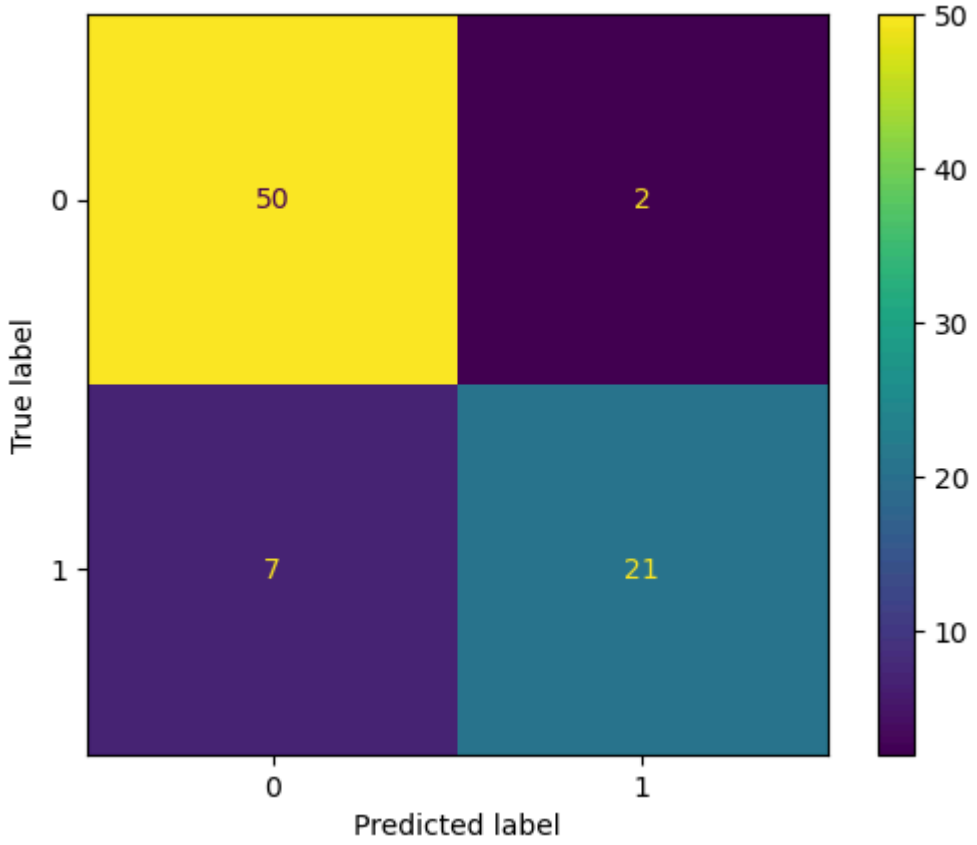
In [25]:
```
model.score(x,y)
```

Out[25]:
```
0.835
```

In [26]:
```
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[50  2]
 [ 7 21]]
```

In [27]:
```
disp=ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=model.clas
disp.plot()
plt.show()
```



In [28]:
```
print(f"TN value is {cm[0][0]}")
print(f"FP value is {cm[0][1]}")
print(f"FN value is {cm[1][0]}")
print(f"TP value is {cm[1][1]}")
```

```
TN value is 50
FP value is 2
FN value is 7
TP value is 21
```

In [40]:
```
print(f"Accuracy score is {accuracy_score(y_test, y_pred)}")
print(f"Error rate is {1-accuracy_score(y_test, y_pred)}")
print(f"Precision score is {precision_score(y_test, y_pred)}")
print(f"Recall score is {recall_score(y_test, y_pred)}")
```

```
Accuracy score is 0.8875
Error rate is 0.11250000000000004
Precision score is 0.9130434782608695
Recall score is 0.75
```

In [32]:
```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.88      0.96      0.92        52
           1       0.91      0.75      0.82        28

    accuracy                           0.89        80
   macro avg       0.90      0.86      0.87        80
weighted avg       0.89      0.89      0.88        80
```

In [ ]: